

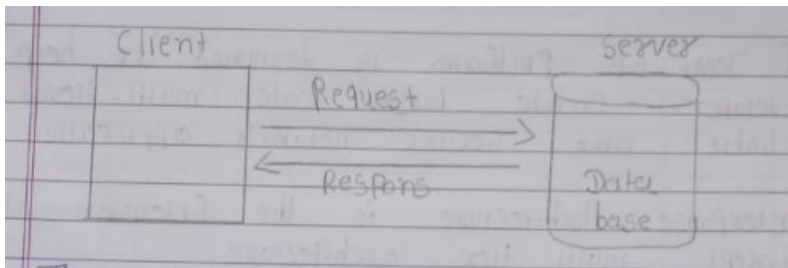
[1] explain enterprise architecture in detail. And Different style of enterprise architecture.

Ans: Enterprise architecture are a software solution that provide business logic and tools to model entire business processes for organizations to improve productivity and efficiency.

- Enterprise Java refers to java applications written for enterprises.
- The leading technology here is java EE which consist of a set of APIs (EJB, JMS, JPA, JSF etc.) and application servers (glassfish, tomcat etc.) which implement these APIs.
 - Enterprise architecture style:
 1. Two-tire Architecture
 2. three-tire Architecture.
 3. N-tire Architecture.

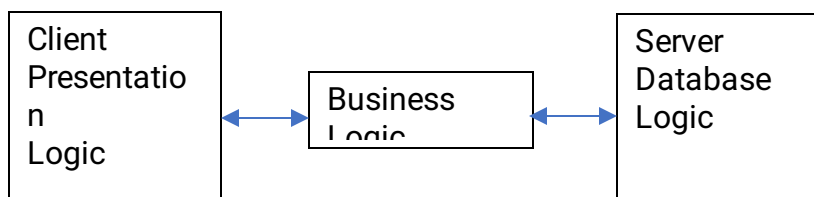
(1) Two-tier architecture: the two-tire architecture is also know as the client server architecture.

- it consists mainly of two-tires: data and client



- the client send a request to the server than server check the available data.
- If the data is available than server send a response to the client that called a client server architecture.
- The application include the presentation and business logic.
- The disadvantage of two tire are limited number of users.

(2) Three tire architecture:



- In 3 tire architecture an application is broken up into three separate logical layers
- Three tire architecture consists: presentation, business Layer, Data layer.

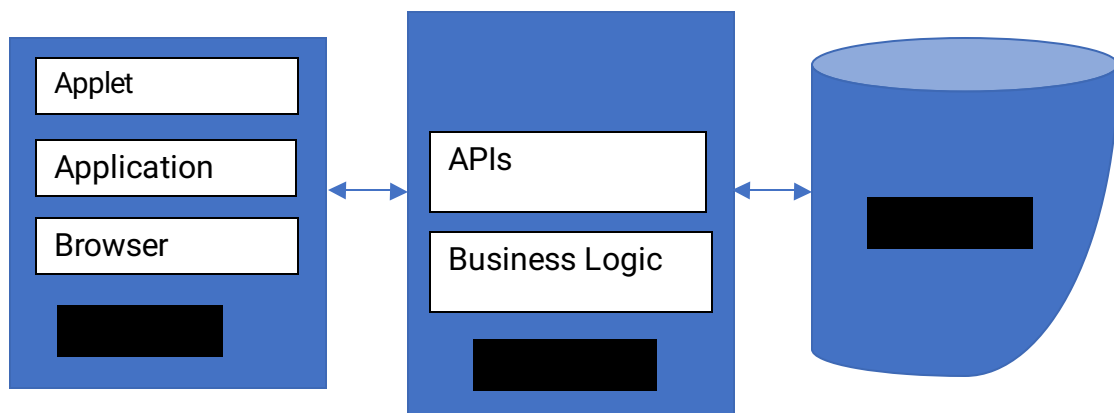
Advantage:

- improves scalability.

Disadvantage:

- increase the complexity

(3) N-Tire Architecture: the N tire Architecture different from the three tire architecture.



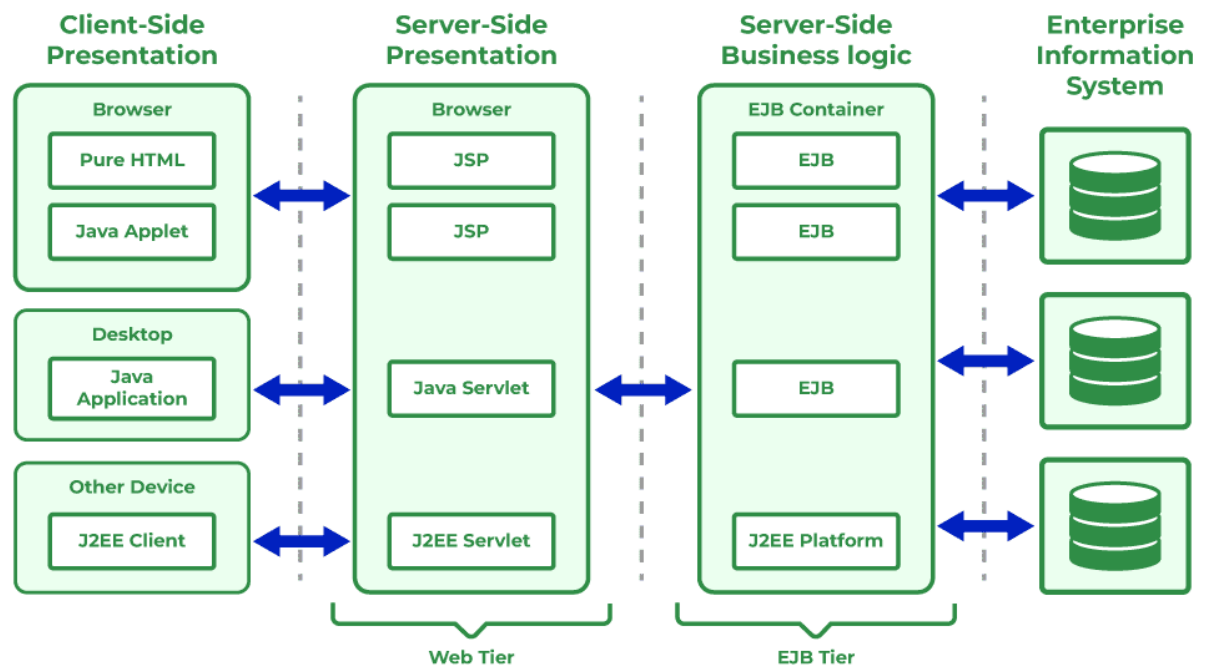
- there is no hard and fast way to define the application layer for n-tier.
- Presentation logic that define the user interface display and how a user's requests are handled.
- Business logic that models the application business rule.
- Data layer when the enterprise data resides.

[2] explain J2EE architecture with container architecture.

Ans: J2EE stand for java 2 enterprise edition, which is a platform for developing and deploying enterprise applications

J2EE architecture also know as Three-tier Architecture.

- Application which is divided into three client tier, server tier and database tier this kind of application is know as three tier architecture



1. Client-tier: Client tier consist of the user interface for user request and print the response.

- This is the tier where the user interacts with the application.
- The client tier communication with the web tier using HTTP or HTTPS
- client tier runs on the client machine.
- Client tier basically uses the following component of client side:

1 Java Applet

2 Java Application

3 Web pages

2. Server-Tier: server side consist of web tier and business tier.

(1) Web Tier: the component of web tier use http to receive request and send responses to the client that reside on the tier.

- The web tier handles the presentation logic and user input validation.
- It also provides access to the EJB using RMI, JNDI or JMS.
- Web tier provides services to the client using HTTP.

- Component of web tier:

1. Servlet

2. JSP

3. JMS

4. JavaMail

5. RMI

(2) Business tier: Business tier consists of the business logic for the J2EE application such as EJB.

- It contains the enterprise beans, such as session beans, entity beans and message driven beans.

3. Database-tier (EIS tier):

- This is the tier where the data is stored and accesses.
- this tier consists of the DBMS/RDBMS.
- The data tier accessed by the EJB using JDBC, JPA or JTA
- It handles the users SQL request and generates proper response based on queries.

[3] List of J2EE Api and explain each.

Ans: J2EE stand for java 2 enterprise edition, which is a platform for developing and deploying enterprise applications.

- All APIs part of the Server Tier in J2EE architecture.
- This App APIs is use to access the database.

(1) JDBC: it is stands for java Database connectivity.

- it is use to connect with Database.
- it is use to establish connection between client and database.

(2) JSP/Servlet: it is use to create dynamic web application.

(3) JMS: it stands for Java Message Service.

- It is use to generate message and send or received message.

(4) JavaMail: it is provide the functionality of email services or it is use to send or receive email.

(5) RMI: it is stand for Remote Method invocations.

- It is use to call the remote method in application.

(6) EJB: it stands for enterprise java bean.

- it is use to create remote components, server component for distributed application.

(7) JavaBean: it is use to create java component in application.

(8) XML: it stands for extensible markup language.

- It is uses to store database.

[4] Write a short note of J2EE Technology.

Ans: there are three technology involves:

1. Component technology
2. Service technology
3. The communication technology

(1) Component technology: these technology are used to hold the most important part of the application the business logic.

- These are three types:

1. JSP Pages
2. Servlets
3. Enterprise JavaBeans

- The component divided into two part:

1. Web Container
2. EJB Container

1. Web Container: these can be category as any component that response to an HTTP request.

JSP, Servlets are most requirement in Web container.

2. EJB Container: the EJB are java programming language server components but contain business logic

Session bean, entity bean and Message Driven Bean are most requirement in Web container.

(2) Service technology: these technology provides the application with supported services to function efficiently.

- some of the J2EE services for application component are managed by the containers.
- It include a different services such as JDBC, JSP, RMI, JTA (Java Transaction API), JAAS (Java Authentication and Authorization Service) and JCA (Java Container Architecture).

(3) The communication technology: This technology provide communication between component and services within an application.

- This protocol are such as:

1. IP
2. HTTP
3. SSL
4. TCP / IP

[5] explain features of J2EE.

Ans: J2EE sand for java 2 enterprise edition, which is a platform for developing and deploying enterprise applications

- Java 2 platform enterprise edition (J2EE) is a set of specification, APIs and technology for developing and deployment multi-tier distributed enterprise applications.
- Features of J2EE:
- J2EE applications are platform independent.
- It create dynamic web application with tools like servlets and JSP.
- It gives scalability for your big java project with EJB.
- Having efficient data management with Java Database connectivity.
- J2EE supports integration of web services by providing robust security.
- Java technology lets programmer handle sensitive data for more securely than they can in C/C++ programs.

[6] what is JDBC Api. Explain JDBC architecture in detail and Explain types of JDBC.

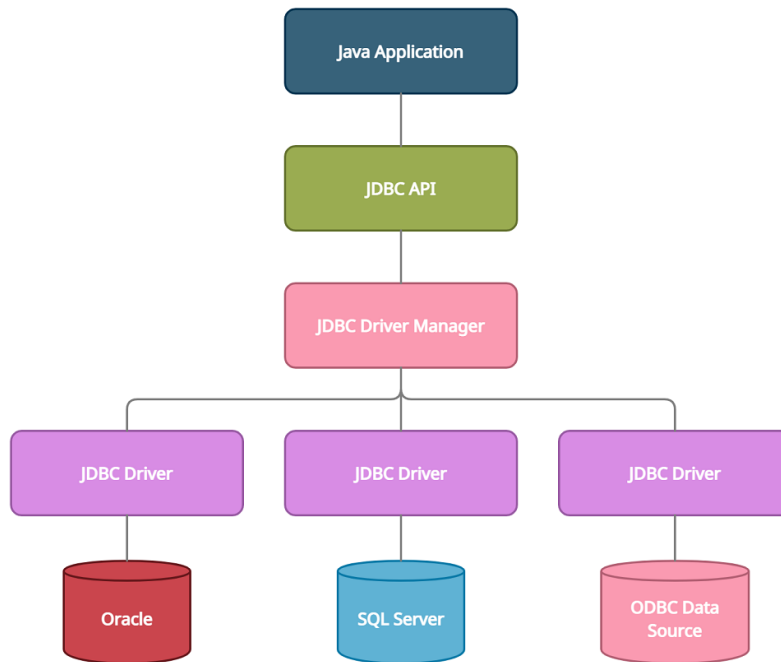
Ans:

JDBC Api: JDBC Api provides universal data access from the java programming language.

- It provides various methods and interfaces for easy communication with the databases.
- JDBC Api comprised of two packages:
 1. Java.sql

2. Javax.sql

- The JDBC API provides the following interfaces and classes –
 1. DriverManager:
 - This class manages a list of database drivers.
 - Matches connection requests from the java application with the proper database driver.
 2. Driver:
 - This interface handles the communications with the database server.
 - You will interact directly with Driver objects very rarely.
 3. Connection:
 - This interface with all methods for contacting a database.
 - The connection object represents communication context, i.e., all communication with the database is through connection object only.
 4. Statement:
 - You use objects created from this interface to submit the SQL statements to the database.
 5. ResultSet:
 - These objects hold data retrieved from a database after you execute an SQL query using Statement objects.
 - It acts as an iterator to allow you to move through its data.
 6. SQLException:
 - This class handles any errors that occur in a database application
- Architecture of JDBC:



1) Application: It is the Java servlet or an applet that communicates with the data source.

2) The JDBC API: It allows the Java programs to perform the execution of the SQL statements and then get the results.

- It provides various methods and interfaces for easy communication with the databases.
- JDBC Api comprised of two packages:
 - Java.sql
 - Javax.sql

A few of the crucial interfaces and classes defined in the JDBC API are the following:

- o Drivers
- o DriverManager
- o Statement
- o Connection
- o CallableStatement
- o PreparedStatement
- o ResultSet
- o SQL data

3) DriverManager: DriverManager plays a crucial role in the architecture of JDBC.

- It uses database-specific drivers to connect the enterprise applications to various databases.

4) JDBC drivers: JDBC Driver is a software component that enables java application to interact with the database.

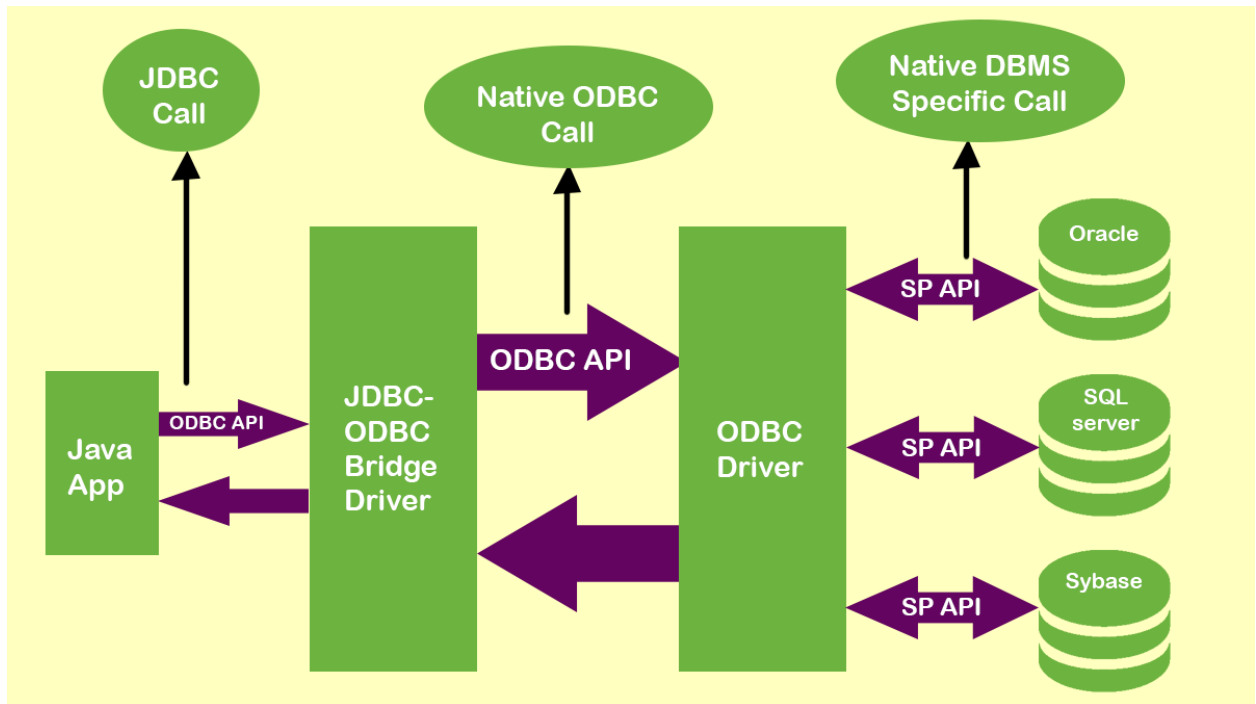
- There are 4 types of JDBC drivers:
 1. JDBC-ODBC bridge driver
 2. Native-API driver (partially java driver)
 3. Network Protocol driver (pure java driver for database middleware)
 4. Native protocol driver (pure java driver directly connected to database)

[7] explain JDBC drivers in detail.

Ans:

JDBC Driver is a software component that enables java application to interact with the database.

- There are 4 types of JDBC drivers:
 5. JDBC-ODBC bridge driver
 6. Native-API driver (partially java driver)
 7. Network Protocol driver (pure java driver for database middleware)
 8. Native protocol driver (pure java driver directly connected to database)
- 1. JDBC-ODBC bridge driver: The JDBC-ODBC bridge driver uses ODBC driver to connect to the database.
 - The JDBC-ODBC bridge driver converts JDBC method calls into the ODBC function calls. This is now discouraged because of thin driver.
 - Oracle does not support the JDBC-ODBC Bridge from Java 8.



- Advantages:

- o easy to use.
- o can be easily connected to any database.

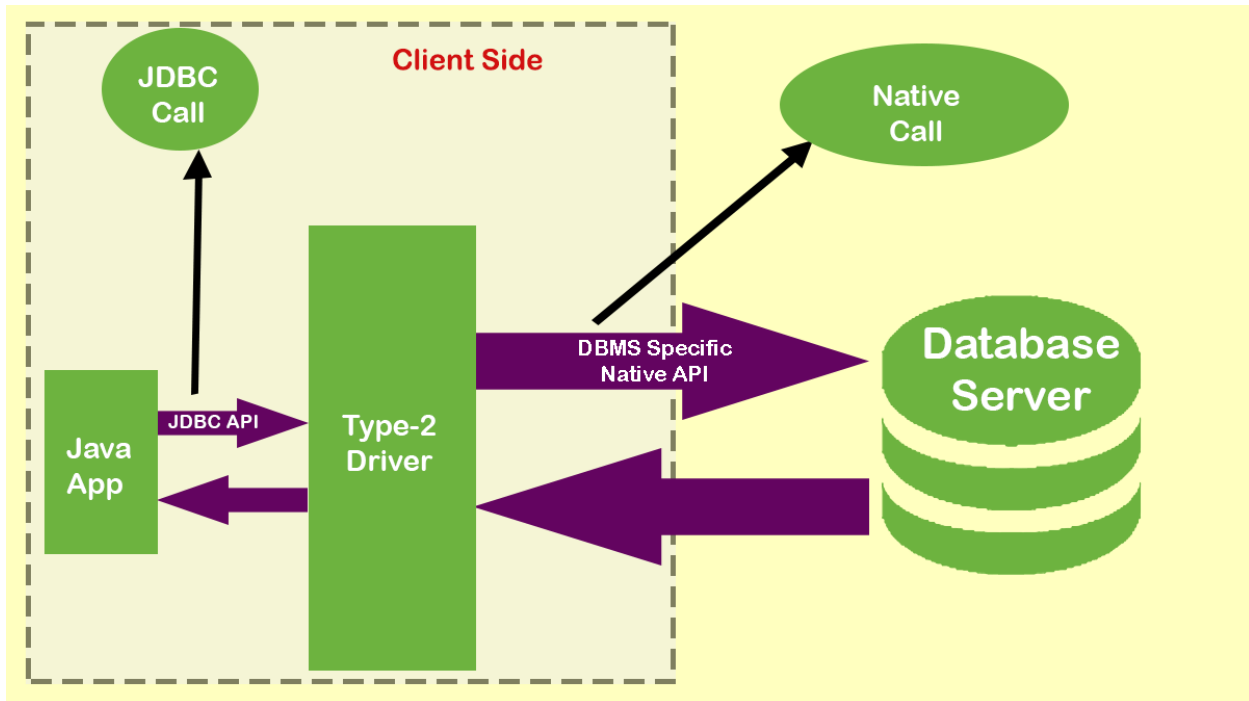
- Disadvantages:

- o Performance degraded because JDBC method call is converted into the ODBC function calls.
- o The ODBC driver needs to be installed on the client machine.

- Native-API driver (partially java driver):

The Native API driver uses the client-side libraries of the database. The driver converts JDBC method calls into

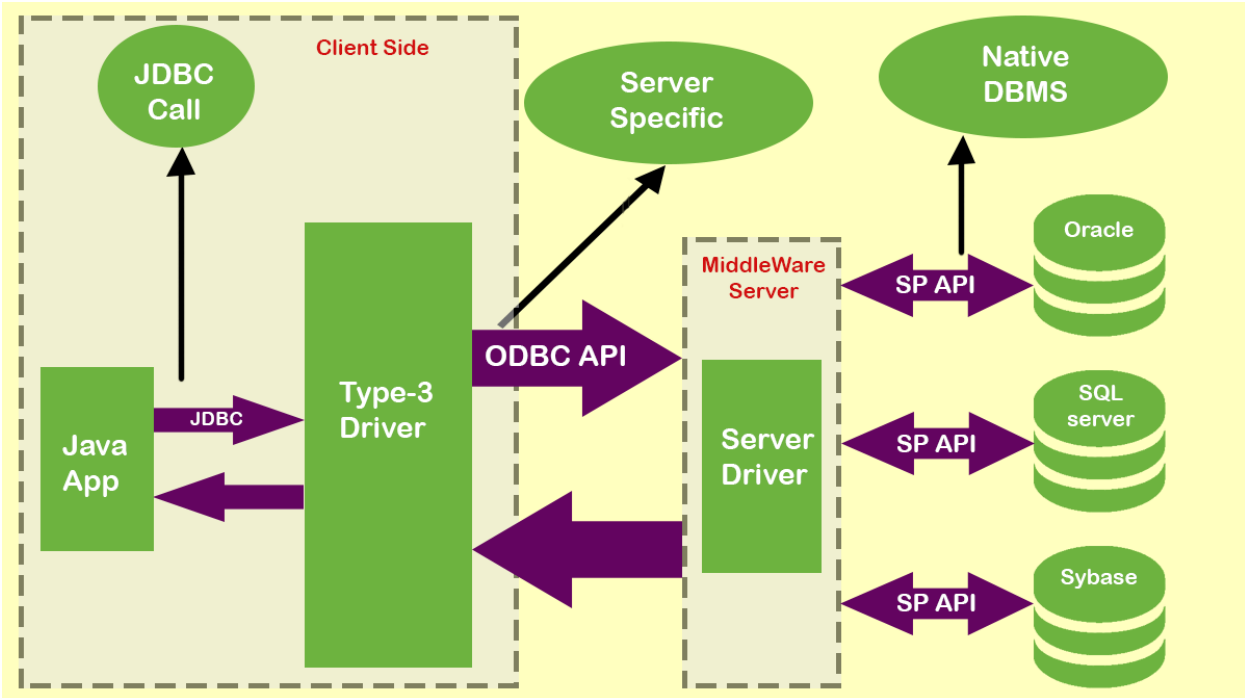
native calls of the database API. It is not written entirely in java.



- Advantage:
 - o performance upgraded than JDBC-ODBC bridge driver.
- Disadvantage:
 - o The Native driver needs to be installed on the each client machine.
 - o The Vendor client library needs to be installed on client machine.

3. Network Protocol driver:

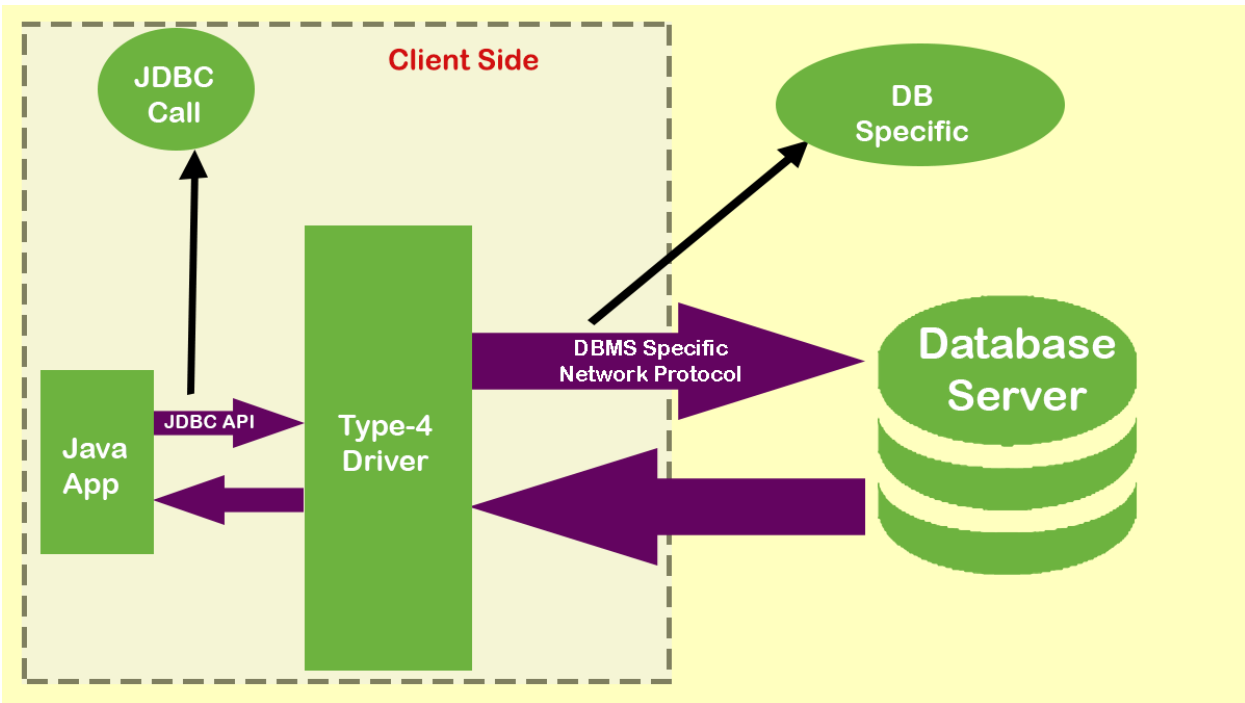
The Network Protocol driver uses middleware (application server) that converts JDBC calls directly or indirectly into the vendor-specific database protocol. It is fully written in java.



- Advantage:
 - o No client side library is required because of application server that can perform many tasks like auditing, load balancing, logging etc.
- Disadvantages:
 - o Network support is required on client machine.
 - o Requires database-specific coding to be done in the middle tier.

Maintenance of Network Protocol driver becomes costly because it requires database-specific coding to be done in the middle tier.

4. Native protocol driver (pure java driver directly connected to database): The Type-4 driver may be a pure Java driver, which implements the database protocol to interact directly with a database.
 - This type of driver does not require any native database library to retrieve the records from the database.
 - In addition, the Type-4 driver converts JDBC calls directly into database specific network calls.



Advantage:

- o Better performance than all other drivers.
- o No software is required at client side or server side.

Disadvantage:

- o Drivers depend on the Database.

[8] explain servlet. Types of the servlet.

Ans: Servlet is a technology which is used to create a web application.

- Servlet is an API that provides many interfaces and classes including documentation.
- Servlet is a web component that is deployed on the server to create a dynamic web page
 - Types of servlets: There are two main types of Servlets.
 1. Generic servlets
 2. HTTP servlets
- 1. Generic servlets: Generic Servlet is a direct division of Servlet type.
 - The servlet is written in this form `javax.servlet.GenericServlet`
 - It provides the implementation of all the methods of these interfaces except the

service method.

- GenericServlet class can handle any type of request so it is protocol-independent.
- You may create a generic servlet by inheriting the GenericServlet class and providing the implementation of the service method.
- Example:

```
import java.io.*;
import javax.servlet.*;

public class First extends GenericServlet{
    public void service(ServletRequest req,ServletResponse res)
    throws IOException, ServletException{

        res.setContentType("text/html");

        PrintWriter out=res.getWriter();
        out.print("<html><body>");
        out.print("<b>hello generic servlet</b>");
        out.print("</body></html>");
    }
}
```

2. HTTP servlets: HTTP Servlet is a subclass of Generic Servlet.

- The servlet is called as javax.servlet.HttpServlet.
- It provides http specific methods such as doGet, doPost, doHead, doTrace etc.
- HttpServlet defines a HTTP protocol specific servlet.
- HttpServlet gives a blueprint for Http servlet and makes writing them easier.
- HttpServlet extends the GenericServlet and hence inherits the properties GenericServlet.

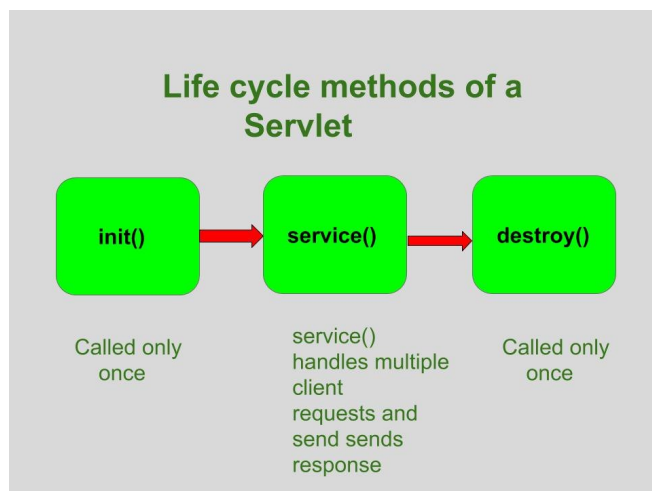
[9] explain servlet life cycle.

Ans: Servlet is a technology which is used to create a web application.

- Servlet is an API that provides many interfaces and classes including

documentation.

- Servlet is a web component that is deployed on the server to create a dynamic web page
- Stage of servlet life cycle:
 1. Loading a servlet
 2. Initialization the servlet
 3. Request handling
 4. Destroying the servlet
- There are three life cycle methods of a Servlet:
 1. `init()`
 2. `service()`
 3. `destroy()`



1. `init()`: servlet object will be initialized by invoking `init()` method.
- Init method is only one time executed.
2. `Service()`: client sent to the request than invoke to the service method.
- Service method handle or serve the client request.
 - In this phase, service method will be invoked and Service method check to the

request type means suppose request are get type request to call DoGet method or request are post type request to call DoPost method.

- Client send to the request than thread is create And this thread handle to the request and service method many time executed this concept is know as multithreading.
3. Destroy(): server are shutdown than servlet object is delete or destroy and destroy method is executed.
- When the server is shutdown, destroy method will be executed and servlet object will be deleted.
 - The destroy() method is called only once.
 - It is called at the end of the life cycle of the servlet.

[10] what is session. Explain session management technique in detail.

Ans: Session is used to save user information momentarily on the server.

- It starts from the instance the user logs into the application and remains till the user logs out of the application or shuts down the machine.
 - In both cases, the session values are deleted automatically.
- There are 4 type in session management technique:
1. Cookies
 2. Hidden from filed
 3. URL rewriting
 4. HTTP Session
1. Cookies: Cookies are small piece of information that is sent by web server in response header and gets stored in the browser cookies.
- Cookie is also client based.
 - Cookie is not a secure method.
- Creating a new Cookie
- ```
Cookie ck = new Cookie("username", name);
```
- Sending Cookies to the client

```
response.addCookie(ck);
```

2. Hidden from filed: We can create a unique hidden field in the HTML and when user starts navigating.

- These fields are not visible directly to the user, but can be viewed using view source option from the browsers.

- Syntax:

```
<input type="hidden" name="java" value="servlet">
```

3. URL Rewriting: We can append a session identifier parameter with every request and response to keep track of the session.

- Disadvantage is, implementing this type of session tracking is tedious.
- Original URL: <http://localhost:8080/servlet/ServletName>
- Rewritten URL: <http://localhost:8080/servlet/ServletName?sessionId=1234>

4. HTTP Session: Servlet API provides Session management through HttpSession interface.

- it is secure method
  - it is server based method.
  - We can get session from HttpServletRequest object using following methods.
1. HttpSession getSession() - This method always returns a HttpSession object. It returns the session object attached with the request, if the request has no session attached, then it creates a new session and return it.
  2. HttpSession getSession(boolean flag) - This method returns HttpSession object if request has session else it returns null.

- Syntax:

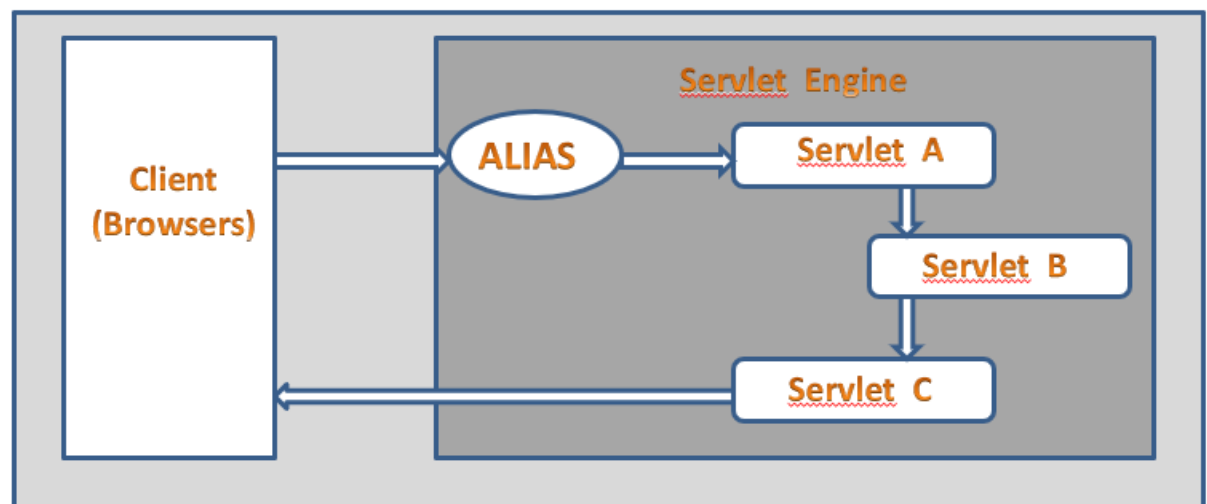
```
HttpSession ses_object = request.getSession(),
ses_object.setAttribute("ses_name", ses_value);
Ses_object.getAttribute("ses_name");
```

[11] explain servlet collaboration in detail.

Ans: Communication between two or more servlet is called servlet collaboration.

- servlets collaborating can pass the shared information directly from one servlet to another through method invocations, as shown earlier.
- The servlet api provides two interfaces namely:

1. `javax.servlet.RequestDispatcher`
  2. `javax.servlet.http.HttpServletResponse`
- the servlet collaborating is two types:
    1. Servlet Chaining.
    2. Request Dispatching
1. Servlet Chaining: Servlet Chaining means multiple servlets are processing a single request is called servlet chaining and a single request is dispatched to multiple servlets.
- Servlet Chaining was initially introduced in the java web server, but was never part of the Servlet API specification.



2. Request Dispatching: The RequestDispatcher interface provides the facility of dispatching the request to another resource it may be html, servlet or jsp.
- This interface can also be used to include the content of another resource also.
  - It is one of the ways of servlet collaboration.
- Servlet collaboration are three methods:
    1. `Forward()`
    2. `Include ()`
    3. `SendRedirect()`
1. `Forward()`: This method forwards a request from a servlet to another servlet on the same server.
- Syntax:

```
// r - requestDispatcher object
```

```
r.forward(req, resp);
```

2. **Include() method:**
3. **sendRedirect() method:** It sends a temporary redirect response to the client using the specified redirect location URL given in the method and clears the buffer.
- **Syntax:**

```
resp.sendRedirect(java.lang.String location)
```

[12] what is JSP. Explain processing of jsp.

Ans: jsp is stand for java server pages.

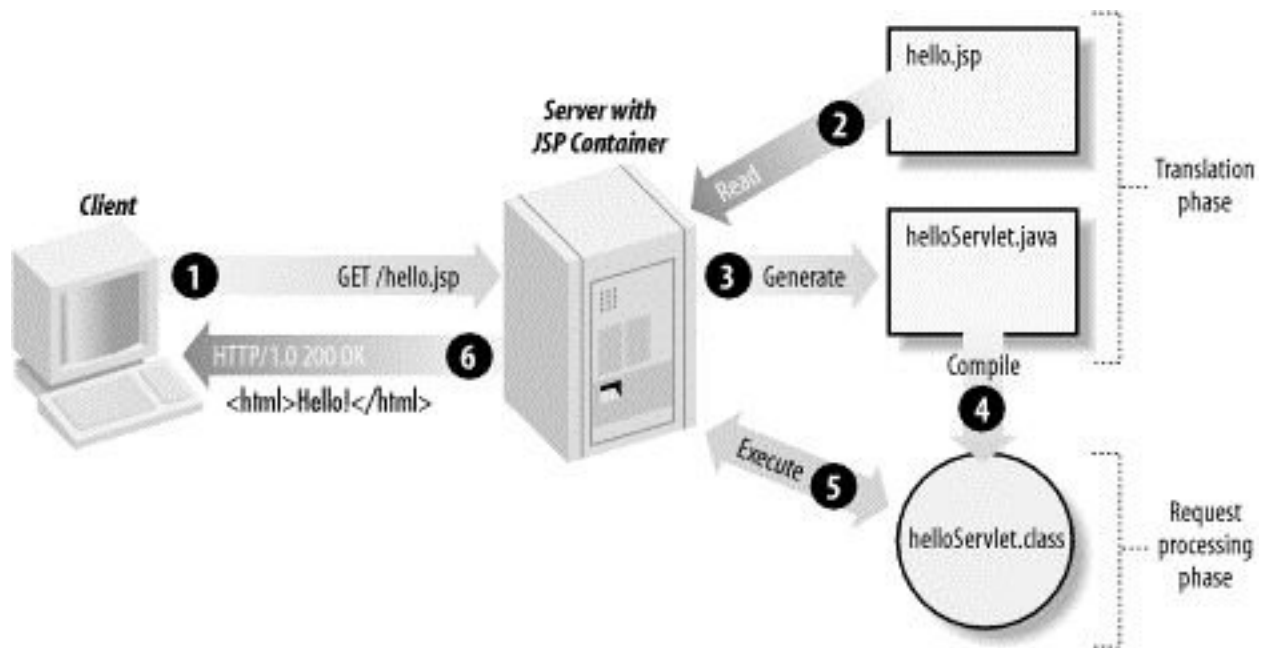
- Java Server Pages (jsp) are built on top of the Java Servlets API, so like Servlets, JSP also has access to all the powerful Enterprise Java APIs, including JDBC, JNDI, EJB, JAXP etc.
- JSP pages can be used in combination with servlets that handle the business logic, the model supported by Java servlet template engines.

- JSP Processing:

The following steps explain how the web server creates the web page using JSP:

- As with a normal page, your browser sends an HTTP request to the web server.
- The web server recognizes that the HTTP request is for a JSP page and forwards it to a JSP engine. This is done by using the URL or JSP page which ends with .jsp instead of .html.
- The JSP engine loads the JSP page from disk and converts it into a servlet content. This conversion is very simple in which all template text is converted to `println()` statements and all JSP elements are converted to Java code that implements the corresponding dynamic behavior of the page.
- The JSP engine compiles the servlet into an executable class and forwards the original request to a servlet engine.
- A part of the web server called the servlet engine loads the Servlet class and executes it. During execution, the servlet produces an output in HTML format, which the servlet engine passes to the web server inside an HTTP response.
- The web server forwards the HTTP response to your browser in terms of static HTML content.
- Finally web browser handles the dynamically generated HTML page inside the HTTP response exactly as if it were a static page.

- All the above mentioned steps can be shown below in the following diagram:



[13] explain jsp scriptlet, jsp declaration, jsp expression and jsp comments.

Ana:

1. JSP scriptlet: A scriptlet can contain any number of JAVA language statements, variable or method declarations that are valid in the page scripting language.

- Following is the syntax of Scriptlet:

```
<% code fragment %>
```

- example for JSP:

```

<html>
 <head><title>Hello World</title></head>
 <body>
 Hello World!

 <%out.println("Your IP address is " + request.getRemoteAddr());%>
 </body>
</html>

```

3. JSP Declarations: A declaration declares one or more variables or methods

that you can use in Java code later in the JSP file.

- You must declare the variable or method before you use it in the JSP file.
- Following is the syntax of JSP Declarations:

```
<%! declaration; [declaration;]+ ... %>
```

Following is the simple example for JSP Declarations:

```
<%! int i = 0; %>
```

```
<%! int a, b, c; %>
```

```
<%! Circle a = new Circle(2.0); %>
```

4. JSP Expression: A JSP expression element contains a scripting language expression that is evaluated and converted to a String where the expression appears in the JSP file.

```
<%= expression %>
```

```
<html>
```

```
<head>
```

```
<title>A Comment Test</title>
```

```
</head>
```

```
<body>
```

```
<p> Today's date: <%= (new java.util.Date()).toLocaleString() %></p>
```

```
</body>
```

```
</html>
```

5. JSP Comments: JSP comment marks text or statements that the JSP container should ignore.

- A JSP comment is useful when you want to hide or "comment out" part of your JSP page.
- Following is the syntax of JSP comments:

```
<%- This is JSP comment -%>
```

There are a small number of special constructs you can use in various cases to insert comments or characters that would otherwise be treated specially. Here's a summary:

Syntax	Purpose
<code>&lt;%-- comment -- %&gt;</code>	A JSP comment. Ignored by the JSP engine.
<code>&lt;!-- comment --&gt;</code>	An HTML comment. Ignored by the browser.
<code>&lt;\%</code>	Represents static <code>&lt;%</code> literal.
<code>%\&gt;</code>	Represents static <code>%&gt;</code> literal.
<code>\'</code>	A single quote in an attribute that uses single quotes.
<code>\"</code>	A double quote in an attribute that uses double quotes.

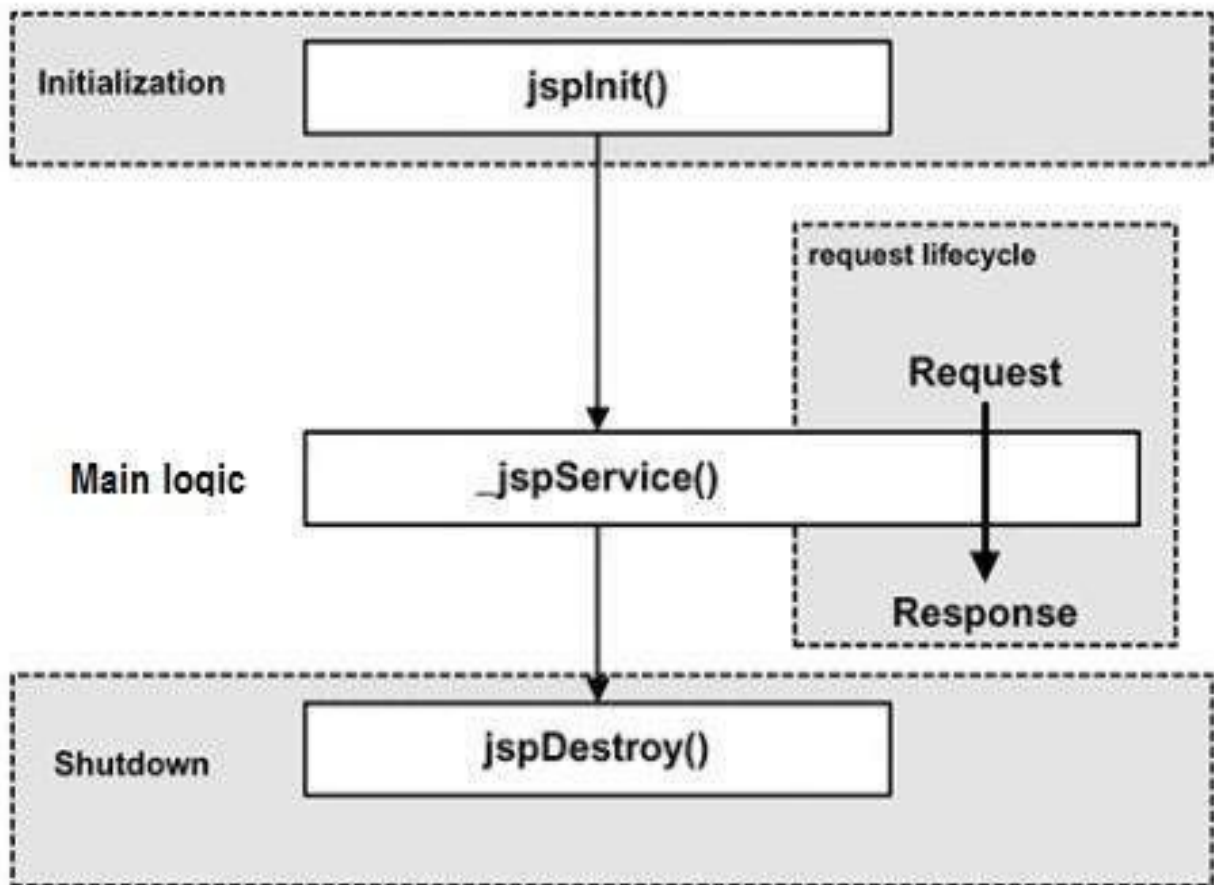
[14] explain JSP life cycle.

Ans: jsp is stand for java server pages.

- Jsp is server-side programming technology that creation of dynamic and platform independent method for building web based applications.
- Java Server Pages (jsp) are built on top of the Java Servlets API, so like Servlets
- JSP also has access to all the powerful Enterprise Java APIs, including JDBC, JNDI, EJB, JAXP etc.

there are certain six phases in which this cycle is divided:

1. Translation phase
2. Compilation phase
3. Class loading
4. Instantiation of servlet object
5. Servlet container calls `jspinit()` method for initialization
6. Servlet container calls `japervice()` for processing the request



1. `Jspinit()`: this method is all object to initialize.
  - `Jspinit` called by the web container when the servlet instance is first created.
2. `Jspservice()`: then init method is completed than automatic execute in service method.
  - `Jspservice()` method is the live statement execution.
  - `Jspservice()` method is called by the web container to process a user request.
3. `Jspdestroy()`: the destroy method is memory space variable is free.
  - `Jsp destroy` called by the web container when the servlet instance is being eliminated.

[15] explain Jsp Directive.

Ans: JSP directives provide directions and instructions to the container, telling it how to handle certain of JSP processing.

- The jsp directives are direction given by web container to servlet.



- It usually has the following form:

```
<%@ directive attribute="value" %>
```

- Different types of jsp directive:

(1) The page Directive: The page directive is used to provide instructions to the container that certain to the current JSP page.

- The page directive defines attributes that apply to an entire JSP page.

- syntax of page directive:

```
<%@ page attribute="value" %>
```

- This directive attribute:

1. Import
2. Language
3. errorPage

- Example:

```
<%@page contentType="text/html" import="java.sql" pageEncoding="UTF-8">
```

```
<html>
```

```
<head>
```

```
<title> jsp page directive</title>
```

```
</head>
```

```
<body>
```

```
 <%
```

```
 Int a=10, b=20, c=a+b;
```

```
 out.println(c);
```

```
 %>
```

```
</body>
```

```
</html>
```

(2) The include Directive: The include directive is used to includes a file during the translation phase.

- The include directive is used to include the contents of any resources it may be jsp file, html file or text file.

- You may code include directives anywhere in your JSP page.

The general usage form of this directive is as follows:

```
<%@ include file="Resource name" >
```

- Example:

```
<%@page contentType="text/html" import="java.sql" pageEncoding="UTF-8">
```

```
<html>
```

```
<head>
```

```
<title> jsp page directive</title>
```

```
</head>
```

```
<body>
```

```
 <% @include file="title.jsp" %>
```

```
</body>
```

```
</html>
```

- (3) The taglib Directive: this directive is used for adding any existing custom tag library with the current JSP page.

- Syntax:

```
<%@ taglib uri="uri" prefix="prefixOfTag" %>
```

- Example:

```
<%@ taglib uri="http://java/core" prefix="c " % >
```

```
<c:forEach var="i" begin="1" end="5">
```

```
<c:out value="$i"/>

```

```
</c:forEach>
```

#### [16] Explain implicit object in JSP.

Ans: This object is directly used in scriptlet that goes in the service method.

- They are created by the container automatically, and they can be accessed using objects.

- A list of the 9 implicit objects is given below:

1. out
2. request
3. response
4. config
5. session
6. application
7. page
8. pageContext
9. exception

1) JSP out implicit object: It is the most used implicit object that comes from `java.servlet.jsp.JspWriter`.

- It's work is to write data into a buffer which would be sent to the client as output.
- It possesses the access to servlet output stream.

Syntax: `out.methodName();`

2) JSP request implicit object: This is an in-built implicit object of `javax.servlet.http.HttpServletRequest`.

- It uses the `getParameter()` method to access the request parameter.

Syntax: `String name=request.getParameter("username");`

3) JSP response implicit object: This is an object from `HttpServletResponse`.

- It is the response given to the user.
- The container creates it for every request.

Ex: `response.sendRedirect("http://www.google.com");`

4) JSP session implicit object: session object is the implicit object under `javax.servlet.http.HttpSession`.

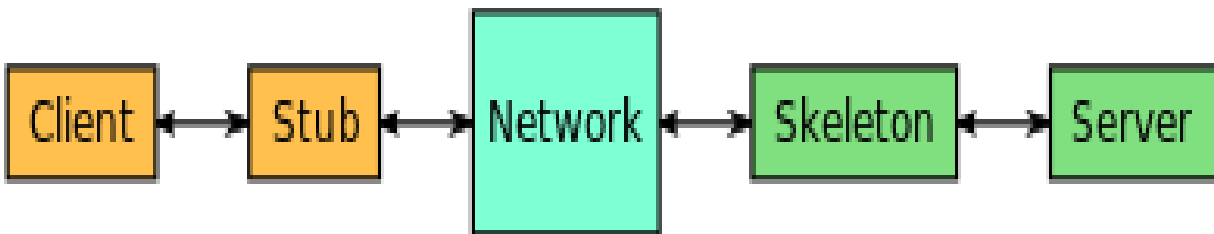
- It is used to get session information as it tracks sessions between client requests.
- Syntax: `session.getAttribute()`

5) JSP Application implicit object: It is an implicit object of `ServletContext` from `javax.servlet.ServletContext` implementation.

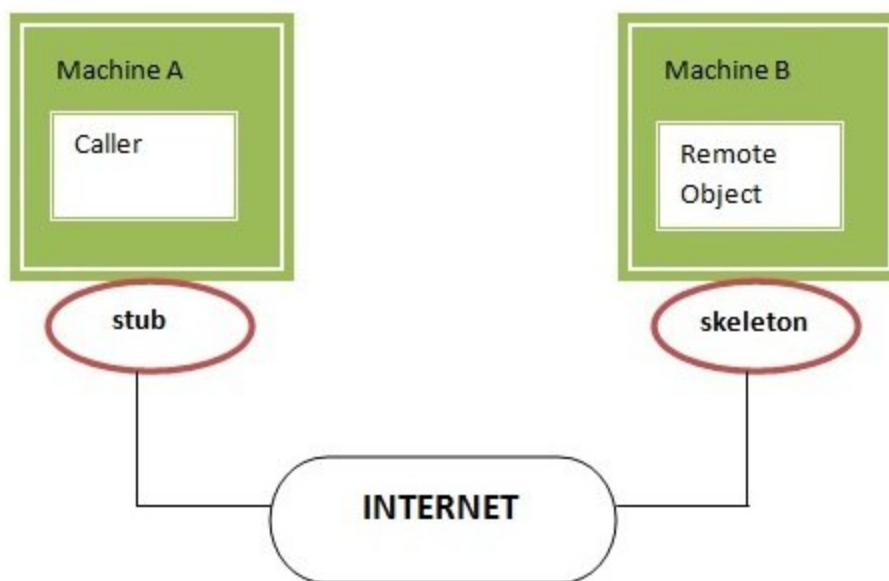
- This object is created by the web container only when an application is deployed.
- Syntax: `application.methodName("value");`

[17] what is RMI. Explain stub and skeleton OR explain RMI architecture.

Ans: The RMI (Remote Method Invocation) is an API that provides a mechanism to create distributed application in java



- The RMI provides remote communication between the applications using two objects stub and skeleton.
- RMI uses stub and skeleton object for communication with the remote object.



1. Stub: The stub is an object that are gateway for the client side.
- All the outgoing requests are routed through it.

- When the caller invokes method on the stub object, it does the following tasks:
- It initiates a connection with remote Virtual Machine (JVM),
- It writes and transmits (marshals) the parameters to the remote Virtual Machine (JVM),
- It waits for the result
- It reads (unmarshals) the return value or exception, and
- It finally, returns the value to the caller.

1. **Skeleton:** The skeleton is an object that are gateway for the server-side object.

- All the incoming requests are routed through it.
- When the skeleton receives the incoming request, it does the following tasks:
- It reads the parameter for the remote method
- It invokes the method on the actual remote object, and
- It writes and transmits (marshals) the result to the caller.

[18] explain step of create RMI.

Ans:

- The is given the 6 steps to write the RMI program.
1. Create the remote interface
  2. Provide the implementation of the remote interface
  3. Compile the implementation class and create the stub and skeleton objects using the rmic tool
  4. Start the registry service by rmiregistry tool
  5. Create and start the client application
  6. Create and start the remote application or server application

a. Create the remote interface:

- Here, we are creating a remote interface that extends the Remote interface.
- There is only one method named add() and it declares RemoteException.

Ex:

```
import java.rmi.*;
```

```
public interface Adder extends Remote{
 public int add(int x,int y) throws RemoteException;
}
```

b. Provide the implementation of the remote interface:

Now provide the implementation of the remote interface. For providing the implementation of the Remote interface, we need to Either extend the UnicastRemoteObject class or use the exportObject() method of the UnicastRemoteObject class

- In case, you extend the UnicastRemoteObject class, you must define a constructor that declares RemoteException.

Ex:

- `import java.rmi.*;`
- `import java.rmi.server.*;`
- `public class AdderRemote extends UnicastRemoteObject implements Adder{`
- `AdderRemote() throws RemoteException{`
- `super();`
- `}`
- `public int add(int x,int y){return x+y;}`
- `}`

c. Compile the implementation class and create the stub and skeleton objects using the rmic tool:

- Next step is to create stub and skeleton objects using the rmi compiler.
- The rmic tool invokes the RMI compiler and creates stub and skeleton objects.
- ex: `rmic AdderRemote`

d. Start the registry service by rmiregistry tool:

- Now start the registry service by using the rmiregistry tool.
- If you don't specify the port number, it uses a default port number.
- In this example, we are using the port number 5000.

- rmiregistry 5000

e. Create and start the client application:

- At the client we are getting the stub object by the lookup() method of the Naming class and invoking the method on this object.
- ```
import java.rmi.*;
```
- ```
public class MyClient{
```
- ```
public static void main(String args[]){
```
- ```
try{
```
- ```
    Adder stub=(Adder)Naming.lookup("rmi://localhost:5000/sonoo");
```
- ```
 System.out.println(stub.add(34,4));
```
- ```
    }catch(Exception e){}
```
- ```
}
```
- ```
}
```

f. Create and start the remote application or server application:

- Now rmi services need to be hosted in a server process. The Naming class provides methods to get and store the remote object.
- In this example, we are binding the remote object by the name sonoo.

```
import java.rmi.*;
import java.rmi.registry.*;
public class MyServer{
public static void main(String args[]){
try{
Adder skeleton=new AdderRemote();
Naming.rebind("rmi://localhost:5000/sonoo",stub);
```

```
}catch(Exception e){System.out.println(e);}
}
}
```

[19] Write a program of RMI for addition of two number.

Ans:

- **SumInterface. JAVA:**

```
import java.rmi.*;
public interface Adder extends Remote{
    public int add(int x,int y) throws RemoteException;
}
```

- **SumServer.Java:**

```
import java.rmi.*;
import java.rmi.server.*;
public class AdderRemote extends UnicastRemoteObject implements Adder{
    AdderRemote() throws RemoteException{
        super();
    }
    public int add(int x,int y){return x+y;}
}
```

- **SumClient.java**

```
import java.rmi.*;
public class MyClient{
    public static void main(String args[]){
        try{
            SumInterface s1=(s1)Naming.lookup("/sum");
            Int s = s1.sum(10,20);
            System.out.println("sum of two number" +s);
        }
    }
}
```



```

    }catch(Exception e){
        System.out.println(e);
    }
}
}

```

- **SumRegister.java**

```

import java.rmi.*;
import java.rmi.registry.*;
public class MyServer{
    public static void main(String args[]){
        try{
            sumServer ss = new sumServer();
            Naming.rebind("/sum",ss);
            System.out.println("the object bind with registry" +ss);
        }catch(Exception e){
            System.out.println(e);
        }
    }
}

```

[20] write a short note of RMI exception.

Ans: When a remote object class is created that extends UnicastRemoteObject, the object is exported, meaning it can receive calls from external Java virtual machines and can be passed in an RMI call as either a parameter or return value.

- An object can either be exported on an anonymous port or on a specified port. For objects not extended from UnicastRemoteObject, the `java.rmi.server.UnicastRemoteObject.exportObject` method is used to explicitly export the object.

Exceptions During RMI Call

| Exception | Context |
|--|--|
| java.rmi.UnknownHostException | Unknown host. |
| java.rmi.ConnectException | Connection refused to host. |
| java.rmi.ConnectIOException | I/O error creating connection. |
| java.rmi.MarshalException | I/O error marshaling transport header, marshaling call header or marshaling arguments. |
| java.rmi.NoSuchObjectException | Attempt to invoke a method on an object that is no longer available. |
| java.rmi.StubNotFoundException | Remote object not exported. |

Exceptions or Errors During Return

| Exception | Context |
|---|--|
| java.rmi.UnmarshalException | <ol style="list-style-type: none"> Corrupted stream leads to I/O or protocol error: <ul style="list-style-type: none"> Unmarshaling return header. Checking return type. Checking return code. Unmarshaling return. Return value class not found. |
| java.rmi.UnexpectedException | An exception not in the method signature occurred, including runtime exceptions on the client. This exception object contains the underlying exception. |
| java.rmi.ServerError | Any error that occurs while the server is executing a remote method. |
| java.rmi.ServerException | Any remote exception that occurs while the server is executing a remote method. See Section A.3.1 for examples. |
| java.rmi.ServerRuntimeException | Any runtime exception that occurs while the server is executing a method, even if the exception is in the method signature. This exception object contains the underlying exception. |

[21] write a code to demonstrate the concept of JSP page.

Ans:

```
<%@page import="java.util.*"%>
<HTML>
<HEAD>
```

```

<TITLE> jsp page </TITLE>
</HEAD>

<BODY>
    <H1> example </H1>

    <% int l; sum=0 %>

    <%
        For(i=1; i<=100; i++)
        {
            Sum = sum + l;
        }
        Out.println("sum is =" +sum);
    %>
</BODY>
</HTML>

```

[22] What is DTD? Explain different types of DTD OR Write short note on XML Validation.

Ans: A Document Type Definition (DTD) describes the tree structure of an XML document.

- It describes the objects such as element, attributes, entities etc. and the relationship of the object.
 - There are two types of DTD:
 1. Internal DTD
 2. External DTD
1. Internal DTD: DTD file create an XML file which is called Internal DTD.

Ex:

```

<?xml version="1.0"?>
<!DOCTYPE note [
    <!ELEMENT note (to,from,heading,body)>
    <!ELEMENT to (#PCDATA)>
    <!ELEMENT from (#PCDATA)>
    <!ELEMENT heading (#PCDATA)>
    <!ELEMENT body (#PCDATA)>
]>
<note>

```

```
<to>Tove</to>
<from>Jani</from>
<heading>Reminder</heading>
<body>Don't forget me this weekend</body>
</note>
```

2. External DTD: If the DTD is declared in an external file.

Ex:

```
<?xml version="1.0"?>
<!DOCTYPE note SYSTEM "note.dtd">
<note>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend!</body>
</note>
```

- And here is the file "note.dtd", which contains the DTD:

```
<!ELEMENT note (to,from,heading,body)>
<!ELEMENT to (#PCDATA)>
<!ELEMENT from (#PCDATA)>
<!ELEMENT heading (#PCDATA)>
<!ELEMENT body (#PCDATA)>
```

[23] What is XML. Explain basic building block in XML.

Ans: The Building Blocks of XML Documents

Seen from a DTD point of view, all XML documents are made up by the following building blocks:

- Elements
- Attributes
- Entities
- PCDATA
- CDATA

1. Elements: Elements are the main building blocks of both XML and HTML documents.

Examples of HTML elements are "body" and "table". Examples of XML elements could be "note" and "message". Elements can contain text, other elements, or be empty. Examples of empty HTML elements are "hr", "br" and "img".

Examples:

```
<body>some text</body>
```

```
<message>some text</message>
```

2. Attributes: Attributes provide extra information about elements.

Attributes are always placed inside the opening tag of an element. Attributes always come in name/value pairs. The following "img" element has additional information about a source file:

```

```

The name of the element is "img". The name of the attribute is "src". The value of the attribute is "computer.gif". Since the element itself is empty it is closed by a "/"

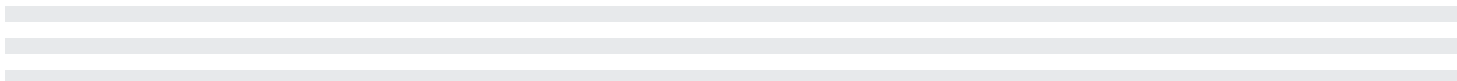
3. Entities: Some characters have a special meaning in XML, like the less than sign (<) that defines the start of an XML tag.

Most of you know the HTML entity: " ". This "no-breaking-space" entity is used in HTML to insert an extra space in a document. Entities are expanded when a document is parsed by an XML parser.

The following entities are predefined in XML:

Entity References

Character



<

<

4. PCDATA: PCDATA means parsed character data.

Think of character data as the text found between the start tag and the end tag of an XML element.

PCDATA is text that WILL be parsed by a parser. The text will be examined by the parser for entities and markup.

Tags inside the text will be treated as markup and entities will be expanded.

However, parsed character data should not contain any &, <, or > characters; these need to be represented by the & < and > entities, respectively.

5. CDATA: CDATA means character data.

CDATA is text that will NOT be parsed by a parser. Tags inside the text will NOT be treated as markup and entities will not be expanded.

[24] Explain Rule for XML document with example.

Ans:

[25] Write a short note on XML element and XML attribute.

Ans:

- XML element:

- An XML element is everything from including the element's start tag to including)the element's end tag.

- Example:

```
<bookstore>
  <book category="CHILDREN">
    <title>Harry Potter</title>
    <author>J K. Rowling</author>
    <year>2005</year>
```

```
<price>29.99</price>
</book>
</bookstore>
```

- XML elements must follow these naming rules:

Names can contain letters, numbers, and other characters

Names cannot start with a number or punctuation character

Names cannot start with the letters xml (or XML, or Xml, etc)

Names cannot contain spaces

Any name can be used, no words are reserved.

- XML Attribute:

- XML elements can have attributes, just like HTML.

- Attributes provide additional information about an element.

```

<a href="demo.asp">
```

- XML Attributes Must be Quoted

- Attribute values must always be quoted. Either single or double quotes can be used.

For a person's gender, the person element can be written like this:

```
<person gender="male">
```

or like this:

```
<person gender ='male'>
```

[26] [explain Java Bean in Detail.](#)

Ans: It's a Component Architecture of Java which allow you to define the Component.

- Java Bean is nothing but the Java Class.

- Java Beans Package:

- Java Bean API makes it possible to write the component s/w in a java programming language.
- This package will be used depending upon the need of the Application (based on function of bean classes).
- Now when its only the use of the property or the Getter & Setter Method at that time you might not even make a use of this package.

- There are two method of Javabeans:

1. Getter method
2. Setter method

1. Getter method:

- It is used for read that property this is called the accessor.

2. Setter method:

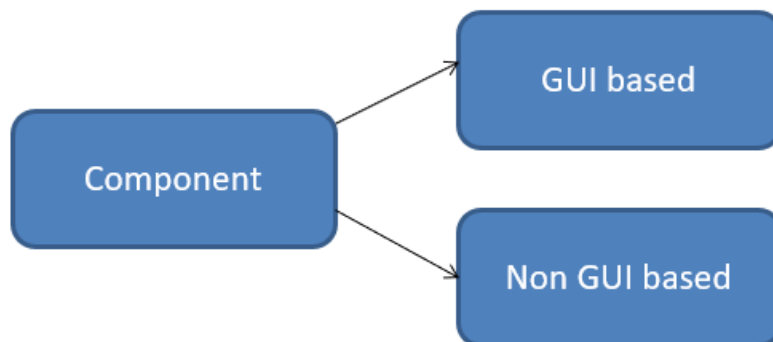
- It is used for write that property this is called the mutator.

- Component of Java Bean:

- The component is nothing but the group of classes.

Example:- JButton

- You can build two types of Components.



- Event: event is one of the action component.

Two types of event:

1. source object
2. listener object

[27] what is EJB. Explain session bean and Entity beans.

Ans: EJB is a server-side component architecture for J2EE.

- An EJB is just a collection of Java classes and XML file, bundled into a single unit.
 - EJB is an essential part of J2EE platform.
 - EJB mainly contain the business logic and business data.
 - Types of EJB: There are three types of EJB:
 1. Session bean
 2. Message Driven bean
 3. Entity bean
1. Session Bean: a session bean is a type of java bean that is used to encapsulate business logic.
 - There are two types of session beans:
 - a. Stateful session bean and
 - b. Stateless session bean.
 - a. Stateful Session bean: it maintains state of a client across multiple requests.
 - b. Stateless Session bean: it does not maintain state of a client between multiple method calls.
 2. Entity Bean: entity bean is a type of java bean that is represent persistent data storage.
 - There are two types of entity bean:
 - a. Bean Managed Persistence: In a bean managed persistence type of entity bean, the programmer has to write the code for database calls.
 - It persists across multiple sessions and multiple clients.
 - b. Container Managed Persistence: Container managed persistence are

enterprise bean that persists across database.

- In container managed persistence the container take care of database calls.

[28] explain Struts technology in java OR describe MVC architecture in detail.

Ans: A strut is the framework for building Java-based Web applications. Using the Model-View-Controller (MVC) design pattern.

- Struts solves many of the problems associated with developing high-performance, business-oriented Web applications that use Java Servlets and JSP.

- **Components of Struts**

- Base Framework :
- provides the core MVC functionality and is comprised of the building blocks for your application.

- JSP Tag Libraries :
- tag libraries enable JSP authors to use HTML-like tags to represent functionality that is defined by a Java class.

HTML

Bean

Logic

Nested

- Tiles Plug-in :
- a rich JSP templating framework that facilitates the reuse of presentation (HTML) code.
- Validator Plug-in :
- provides a rich framework for performing data validation on both the server side and client side (browser).
- **MVC:** MVC (Model, View, Controller) Architecture is the design pattern that is used to build the applications.
- This architectural pattern was mostly used for Web Applications

- **Model-View-Controller Architecture**

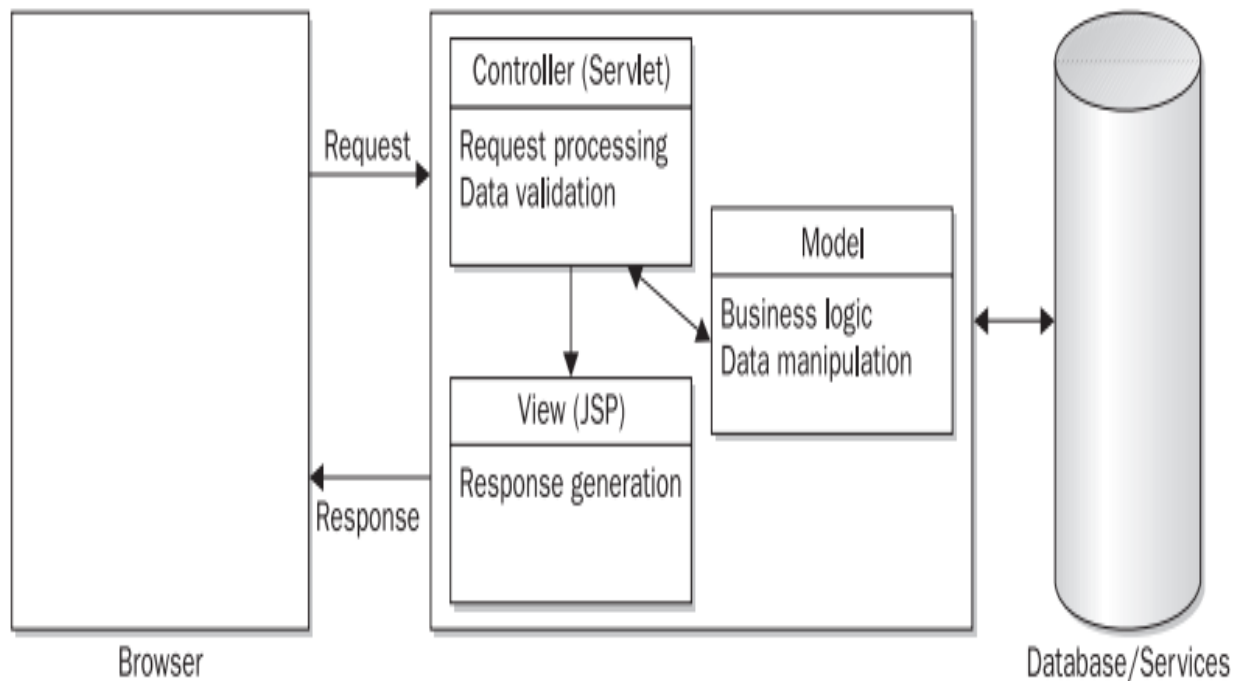
Model: The backend that contains all the data logic

View: The frontend or graphical user interface (GUI)

Controller: The brains of the application that controls how data is displayed

- MVC architecture:

- In the MVC architecture, a central servlet, known as the Controller, receives all requests for the application.
- The Controller then processes the request and works with the Model to prepare any data needed by the View (which is usually a JSP) and forwards the data to a JSP.
- The JSP then uses the data prepared by the Controller to generate a response to the browser.
- Business and presentation logic are separated from each other.



[29] explain ScrollableResultSet object.

Ans:

the object of ResultSet maintains a cursor pointing to a row of a table. Initially, cursor points to before the first row.

But we can make this object to move forward and backward direction by passing either TYPE_SCROLL_INSENSITIVE or TYPE_SCROLL_SENSITIVE in createStatement(int,int) method as well as we can make this object as updatable by:

```
Statement stmt = con.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
ResultSet.CONCUR_UPDATABLE);
```

* Commonly used methods of ResultSet interface

1) public boolean next():	is used to move the cursor to the one row next from the current position.
2) public boolean previous():	is used to move the cursor to the one row previous from the current position.
3) public boolean first():	is used to move the cursor to the first row in result set object.
4) public boolean last():	is used to move the cursor to the last row in result set object.
5) public boolean absolute(int row):	is used to move the cursor to the specified row number in the ResultSet object.
6) public boolean relative(int row):	is used to move the cursor to the relative row number in the ResultSet object, it may be positive or negative.
7) public int getInt(int columnIndex):	is used to return the data of specified column index of the current row as int.
8) public int getInt(String columnName):	is used to return the data of specified column name of the current row as int.
9) public String getString(int columnIndex):	is used to return the data of specified column index of the current row as String.
10) public String getString(String columnName):	is used to return the data of specified column name of the current row as String.

- Example of Scrollable ResultSet

Let's see the simple example of ResultSet interface to retrieve the data of 3rd row.

1. `import java.sql.*;`
2. `class FetchRecord{`
3. `public static void main(String args[])throws Exception{`
- 4.
5. `Class.forName("oracle.jdbc.driver.OracleDriver");`
6. `Connection con=DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe","system","oracle");`
7. `Statement stmt=con.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,ResultSet.CONCUR_UPDATABLE);`
8. `ResultSet rs=stmt.executeQuery("select * from emp765");`

- 9.
10. *//getting the record of 3rd row*
11. `rs.absolute(3);`
12. `System.out.println(rs.getString(1)+" "+rs.getString(2)+" "+rs.getString(3));`
- 13.
14. `con.close();`
15. `}}`

[\[30\]](#) explain JavaMail in detail.

Ans:

The JavaMail is an API that is used to compose, write and read electronic messages (emails).

The JavaMail API provides protocol-independent and platform-independent framework for sending and receiving mails.

Protocols used in JavaMail API

There are some protocols that are used in JavaMail API.

- o SMTP (Simple mail transfer protocol)
- o POP (post office protocol)
- o IMAP (internet message access protocol)
- o MIME (multiple internet mail extension)
- o NNTP (network news transfer protocol)
- o and others
- You must know following classes before you start
 - Session
 - Message
 - Address
 - Transport
 - Store
 - Folder

// chopda mathi karvo

[31] explain JMS.

